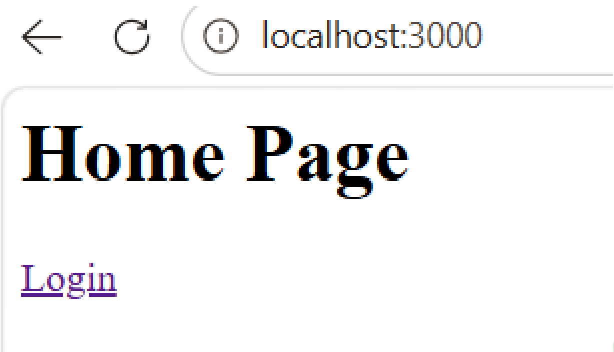
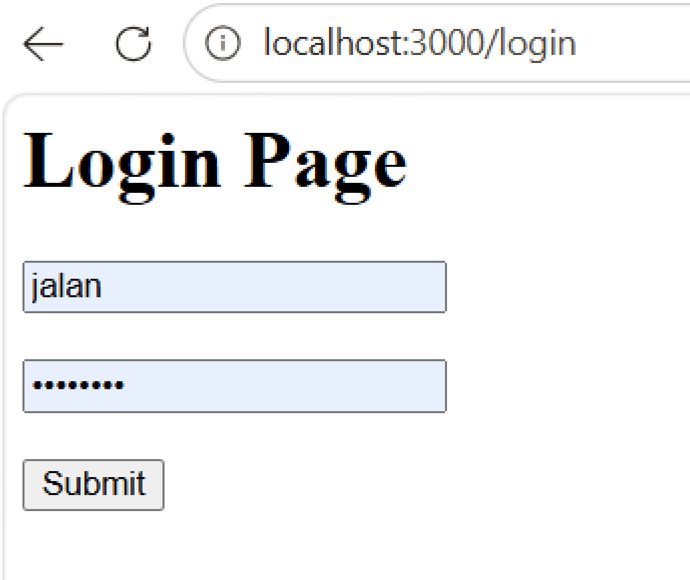
Experiment. Build a web application using Express.js that demonstrates routing and middleware functionality.

Program:

```
const express = require("express");
const app = express();
// Middleware to parse form data
app.use(express.urlencoded({ extended: true }));
// Optional: parse JSON data
app.use(express.json());
// Middleware with next()
app.use((req, res, next) => {
  console.log("Middleware executed");
  next();
});
// GET → Home
app.get("/", (req, res) => {
  res.send(`
    <h1>Home Page</h1>
    <a href="/login">Login</a>
  `);
});
// GET → Login form
app.get("/login", (req, res) => {
  res.send(`
    <h1>Login Page</h1>
    <form method="POST" action="/login">
      <input type="text" name="username" placeholder="Enter username" /><br><br>
      <input type="password" name="password" placeholder="Enter password" /><br><br>
      <button type="submit">Submit</button>
    </form>
  `);
});
// POST → Handle input
app.post("/login", (req, res, next) => {
  const { username, password } = req.body;
  // Validation
  if (!username || !password) {
    return res.send("<h3>All fields are required</h3>");
  }
  console.log("Data received:", req.body);
  next();
}, (req, res) => {
  res.send(`
    <h1>Data Received </h1>
    <p>Username: ${req.body.username}</p>
  `);
});
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

OUTPUT:


localhost:3000
Home Page
Login


localhost:3000/login
Login Page
jalan
Submit


localhost:3000/login
Data Received
Username: jalan

Laboratory Record of Enterprise Application Development Lab

Roll No: 160124737313
Exp No: 11
Sheet No:
Date:22-04-2026

Experiment: Develop a RESTful API using Express.js to manage student data

Program:

```
const express = require('express');
const app = express();
const PORT = 3000;
// Middleware to parse JSON
app.use(express.json());
app.get("/", (req, res) => {
  res.send("Welcome to the Student API!");
});
// In-memory database (for demo)
let students = [
  { id: 1, name: "Ravi", age: 20 },
  { id: 2, name: "Sita", age: 21 }
];
let nextId = 3;
 //GET → Fetch all students
app.get("/students", (req, res) => {
  res.json(students);
});
 //POST → Add new student
 app.post("/students", (req, res) => {
  const { name, age } = req.body;
  if (!name || !age) {
    return res.status(400).json({ message: "Name and age required" });
  }
  const newStudent = {
    id: nextId++,
    name,
    age
  };
  students.push(newStudent);
  res.status(201).json({
    message: "Student added",
    student: newStudent
  });
});
  //PUT → Update student
app.put("/students/:id", (req, res) => {
  const id = parseInt(req.params.id);
  const { name, age } = req.body;
  const student = students.find(s => s.id === id);
  if (!student) {
    return res.status(404).json({ message: "Student not found" });
  }
  if (name) student.name = name;
  if (age) student.age = age;
  res.json({
    message: "Student updated",
    student
  });
});
```

```
// DELETE → Remove student
app.delete("/students/:id", (req, res) => {
  const id = parseInt(req.params.id);
  const index = students.findIndex(s => s.id === id);
  if (index === -1) {
    return res.status(404).json({ message: "Student not found" });
  }
  const deleted = students.splice(index, 1);
  res.json({
    message: "Student deleted",
    student: deleted[0]
  });
});
//start server
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

OUTPUT:

GET:

Status: 200 OK Size: 65 Bytes Time: 14 ms


```
[
  {
    "id": 1,
    "name": "Ravi",
    "age": 20
  },
  {
    "id": 2,
    "name": "Sita",
    "age": 21
  }
]
```


POST:

Status: 201 Created Size: 71 Bytes Time: 17 ms


```
{
  "message": "Student added",
  "student": {
    "id": 3,
    "name": "sriram",
    "age": 18
  }
}
```

PUT:

Status: 200 OK Size: 72 Bytes Time: 31 ms

```
{
  "message": "Student updated",
  "student": {
    "id": 1,
    "name": "geeta",
    "age": 20
  }
}
```

DELETE:

Status: 200 OK Size: 71 Bytes Time: 11 ms

```
{
  "message": "Student deleted",
  "student": {
    "id": 2,
    "name": "Sita",
    "age": 21
  }
}
```

Experiment. Implement user authentication using JSON Web Token (JWT) in an Express.js application.

Program:

```
const express = require("express");
const jwt = require("jsonwebtoken");
const app = express();
app.use(express.json());
const PORT = 3000;
const SECRET_KEY = "secret123"; // use env variable in real apps
const user = {
  id: 1,
  username: "admin",
  password: "1234"
};
app.post("/login", (req, res) => {
  const { username, password } = req.body;
  // Validate user
  if (username !== user.username || password !== user.password) {
    return res.status(401).json({ message: "Invalid credentials" });
  }
  const token = jwt.sign(
    { id: user.id, username: user.username },
    SECRET_KEY,
    { expiresIn: "1h" }
  );
  res.json({
    message: "Login successful",
    token
  });
});
function authenticateToken(req, res, next) {
  const authHeader = req.headers["authorization"];
  // Extract token (Bearer TOKEN)
  const token = authHeader && authHeader.split(" ")[1];
  if (!token) {
    return res.status(401).json({ message: "Access denied. No token provided" });
  }
  try {
    const decoded = jwt.verify(token, SECRET_KEY);
    req.user = decoded;
    next();
  } catch (err) {
    return res.status(403).json({ message: "Invalid or expired token" });
  }
}
app.get("/dashboard", authenticateToken, (req, res) => {
  res.json({
    message: "Welcome to Dashboard",
    user: req.user
  });
});
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
```

```
});
```

OUTPUT:

Status: 200 OK Size: 204 Bytes Time: 21 ms Response


```json
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
      .eyJpZCI6MSwidXNlcm5hbWUiOiJhZG1pbiIsImlhdCI6MTc3NzIxNDQ5NywiZXhwIjoxNzc3MjE4MDk3fQ
      .IC4BOhQ6ugiodt84DEh-5NsgzClxNSS3hjlUngD1fu0"
}
```